# IOU Exchange

**IOUX DEPLOYMENT GUIDE — FOR INFRASTRUCTURE SETUP (VPS / STAGING → PRODUCTION)**

**For** David Sule — infrastructure deploy
**From** Alfred Mulinge — IOUX application
**Date** 15 September 2026
**Note** No production secrets in this PDF

**Document ID:** IOUX-DEPLOY-SULE-001

**Version:** 1.0

**Date:** 15 September 2026

**Prepared for:** David Sule (infrastructure / AfyaX-side engineering)

**Prepared by:** Alfred Mulinge (IOUX application)

**Product:** IOU Exchange (IOUX) — `ioux.africa`

**Related:** Partner API Handbook `IOUX-PARTNER-API-001` · Infra proposal (Namecheap VPS, aligned 14 Sep 2026)

> **Shareable with Sule.** Contains **no production secrets**.
> Secrets, API keys, and DB passwords are exchanged only on a secure channel.

---

## 1. Purpose of this document

This is the practical deploy guide for standing up IOUX infrastructure as agreed:

- Start on a **Quasar-class** (or equivalent) VPS — **not** Pulsar 2 GB
- Keep **current production** ( `uzimax.onrender.com` ) running for Mint/Kenbright and AfyaX trials
- Use the new VPS first as **staging / test**
- Cut over to production **only after validation**
- Design so a later move to **Render** (or another managed host) stays straightforward

This document tells you **what the IOUX app actually needs** to build, run, and harden — so your VPS setup matches the repo, not a generic Node template.

---

## 2. What we agreed (infra decisions)

| DECISION | AGREEMENT |
|---|---|
| Initial host class | Quasar-class (≈ 4 vCPU / 6 GB) or equivalent — avoid 2 GB for app + workers |
| Current prod | Stay on existing Render deploy during active PoCs |
| New VPS role first | **Staging** → validate app, Nginx, TLS, DB connectivity, webhooks |
| Database | Prefer **managed Postgres** (current: Supabase) even on VPS app hosts — do not force full DB onto a small box unless justified |
| Redis | **Required in production** for rate limiting ( `REDIS_URL` ) |
| Email | Keep off the VPS if possible (e.g. Resend / existing mail DNS); preserve MX on cPanel when pointing A records |
| Long-term | VPS for early cost/control; keep a clean path to Render later if managed ops preferred |
| Still to lock on call | Who procures/pays VPS · who owns day-to-day ops · staging hostname · cutover checklist & rollback |

---

## 3. Application stack (from the IOUX repository)

Confirm these before installing packages on the server.

| ITEM | VALUE |
|---|---|
| Runtime | **Node.js ≥ 20** (deploy pin used: **20.18.0**) |
| Package manager | **npm** (use the repo lockfile — `package-lock.json`) |
| App shape | Single process: **Express API + SPA** (`tsx server/index.ts`) |
| Frontend build | Vite → static `dist/` served by the same Node process in current prod pattern |
| Database | **PostgreSQL** (Drizzle ORM · `drizzle-kit push` / migrations) |
| Cache / rate limit | **Redis** via `REDIS_URL` (`ioredis`) |
| Process supervisor | **PM2** or **systemd** (your choice on VPS) |
| Edge | **Nginx** reverse proxy + TLS (Let's Encrypt / Certbot) |
| Health check | `GET /api/v1/health` |

### 3.1 Key npm scripts

| SCRIPT | PURPOSE |
|---|---|
| `npm install` | Install deps from lockfile |
| `npm run build` | Production frontend build (`vite build`) |
| `npm run start:api` | Start API + SPA (`tsx server/index.ts`) |
| `npx drizzle-kit push` | Apply schema to Postgres |
| `npm run db:rls` | Apply RLS policies (as used on Render build) |
| `npm run configure:afyax` | Wire AfyaX integration row (when credentials ready) |
| `npm test` | Unit tests |

**Render reference build/start (current prod pattern):**

```
build:  npm install && npx drizzle-kit push && npm run db:rls
start:  npm run start:api
health: /api/v1/health
```

Mirror this on the VPS (optionally split migrate vs start if you prefer controlled releases).

## 4. Target architecture (staging first)

```
Internet
   │
   ▼
DNS  (staging.ioux.africa  →  VPS public IP)
   │
   ▼
Nginx  :443 TLS  →  proxy_pass  http://127.0.0.1:8787
   │
   ▼
Node IOUX  (Express + SPA)  PORT=8787
   │
   ├── PostgreSQL  (prefer managed Supabase / pooler URL)
   └── Redis       (local on VPS or managed)
```

**Do not** expose Postgres or Redis publicly.

**Do not** expose Node port 8787 publicly — only Nginx 80/443.

### 4.1 Suggested hostnames

| ENV | EXAMPLE | NOTES |
|---|---|---|
| Current production | `https://uzimax.onrender.com` | Keep until cutover |
| Staging (new VPS) | `https://staging.ioux.africa` | First deploy target |
| Future production | `https://app.ioux.africa` / `https://api.ioux.africa` | After validation |

Email DNS (MX / SPF / DKIM / DMARC) must stay with the mail provider when changing A records.

---

## 5. Server provisioning checklist (Phase 1–3)

### Phase 1 — Base server

☐ Provision Quasar-class (or agreed) Ubuntu LTS VPS
☐ Create non-root deploy user + SSH keys
☐ Disable password/root SSH as agreed
☐ UFW: allow `22` (restricted), `80`, `443` only
☐ Optional: fail2ban on SSH
☐ System updates applied

### Phase 2 — Runtime

☐ Install Node **20.x** (nodesource or nvm — pin 20.18.x preferred)
☐ Install `git`, build tools if needed
☐ Install **Redis** (if running locally) and bind to localhost
☐ Confirm `node -v` ≥ 20 and `npm -v`

### Phase 3 — Nginx + TLS

☐ Install Nginx
☐ Point staging DNS A record to VPS
☐ Certbot / Let's Encrypt for staging hostname
☐ HTTP → HTTPS redirect
☐ Proxy `/` and `/api` to `127.0.0.1:8787` with sensible timeouts
☐ Do **not** put cPanel on this VPS for the app host

---

## 6. Application deploy steps (Phase 4)

Run as the deploy user from the app directory (example `/var/www/ioux`).

```
# 1. Get code (private repo access required)
git clone <IOUX_REPO_URL> ioux
cd ioux
git checkout main   # or agreed release tag

# 2. Dependencies
npm ci

# 3. Environment file (never commit)
cp .env.example .env.production   # or create .env
# Fill secrets via secure channel — see §7

# 4. Frontend production build
npm run build

# 5. Database schema (against staging DATABASE_URL)
npx drizzle-kit push
npm run db:rls

# 6. Process supervision (example PM2)
npm install -g pm2
pm2 start "npm run start:api" --name ioux-api
pm2 save
pm2 startup
```

### 6.1 Verify

```
curl -s https://staging.ioux.africa/api/v1/health
# expect: { "status": "ok", ... "db": "up" }
```

☐ Portal loads over HTTPS
☐ Login works against staging DB
☐ Admin → Integrations reachable
☐ Logs rotating (journald / PM2 logs / logrotate)

---

## 7. Environment variables (names only — values via secure channel)

Minimum for a working staging deploy:

| VARIABLE | PURPOSE |
|---|---|
| `NODE_ENV` | `production` |
| `PORT` | `8787` (behind Nginx) |
| `DATABASE_URL` | Postgres connection (prefer pooler; SSL as required) |
| `REDIS_URL` | Redis for rate limiting |
| `JWT_SECRET` | Access tokens |
| `JWT_REFRESH_SECRET` | Refresh tokens |
| `COOKIE_SECURE` | `true` behind HTTPS |
| `PORTAL_URL` / `SITE_URL` | Public staging URLs |
| `EMAIL_PROVIDER` / `RESEND_API_KEY` / `EMAIL_FROM` | Outbound email |
| `AFYAX_WEBHOOK_SECRET` | Inbound payment-update HMAC |
| `IOUX_WEBHOOK_SECRET` | Outbound envelope signing if used |
| `AFYAX_SANDBOX_BASE_URL` | e.g. `https://manager.smplystore.com` |

| VARIABLE | PURPOSE |
|---|---|
| `AFYAX_PRODUCTION_BASE_URL` | e.g. `https://vendor.afyax.health` |
| `AFYAX_DEFAULT_PAYMENT_METHOD` | usually `bank` |

Optional / as needed: S3 document storage keys, SMS provider keys, platform API bootstrap vars.

**Rules:**

- Never put production `.env` in git or WhatsApp
- Separate staging vs production secrets
- Rotate after any accidental exposure

---

## 8. Redis, workers, storage

| COMPONENT | REQUIREMENT |
|---|---|
| Redis | **Yes** for production-like staging ( `REDIS_URL` ). Rate limiter expects it when `NODE_ENV=production` . |
| Background workers | No separate worker process required for core AfyaX path today; webhook delivery runs in-app. Revisit if queues are split later. |
| File storage | Local `STORAGE_LOCAL_PATH` or S3-compatible — agree before go-live if documents must persist across redeploys |

---

## 9. AfyaX integration on the new environment (Phase 5–6)

After staging app is healthy:

1. Configure AfyaX purchase webhook URL on IOUX Integrations (sandbox first).
2. Issue / confirm platform API key for AfyaX (secure handoff to Sule's vault).
3. Confirm HMAC secret for `POST /webhooks/payment-update` .
4. Whitelist VPS **outbound** IP on AfyaX purchase endpoint (if IP allowlisting is used).
5. Run one joint UAT IOU using Partner Handbook §10 script.
6. Confirm `webhook_deliveries` shows 2xx (or documented business 4xx).

Current production for trials remains Render until cutover is approved.

---

## 10. Staging validation → production cutover (Phase 7)

### 10.1 Staging acceptance (before touching prod DNS)

☐ Health green for 24–48h under light load
☐ TLS auto-renewal tested
☐ DB backups verified (managed DB restore test or VPS backup restore)
☐ Redis up; no rate-limit fallback surprises
☐ AfyaX sandbox purchase path tested
☐ Monitoring / alerts (uptime + disk + RAM + 5xx)
☐ Rollback plan written (point DNS back / keep Render warm)

### 10.2 Production cutover (only when approved)

☐ Freeze risky deploys briefly
☐ Final env parity check
☐ Switch DNS A records with low TTL prepared in advance
☐ Watch health, logs, webhook deliveries

☐ Keep previous host available for quick rollback

## 11. Ops ownership (to confirm)

| AREA | PROPOSED DEFAULT | CONFIRM |
|---|---|---|
| VPS procurement / billing | UzimaX commercial | ☐ |
| Linux / Nginx / TLS / firewall | Sule (infra) | ☐ |
| App releases / migrations | Alfred (IOUX app) with Sule access | ☐ |
| DB backups (managed) | Owner of Supabase project | ☐ |
| Secrets vault | Shared secure channel + vault | ☐ |
| Incident first response | Agree primary on-call | ☐ |

## 12. Security baseline

- Non-root deploy user; SSH keys only
- Firewall: 22/80/443 only (22 restricted by IP if possible)
- Secrets outside git
- DB and Redis localhost / private network only
- HTTPS everywhere public
- Patch Ubuntu / Node on a schedule
- Separate staging and production credentials

## 13. Cost / sizing reminder

| OPTION | NOTES |
|---|---|
| Quasar-class VPS | Agreed starting class for comfort (CPU/RAM headroom) |
| Managed Postgres | Recommended vs stuffing DB on the same small box |
| Render | Remains valid later if managed ops outweigh VPS cost |

Do **not** migrate production mid-PoC to save a month of hosting cost.

## 14. What Alfred provides vs what Sule provides

| ALFRED / IOUX APP | SULE / INFRA |
|---|---|
| Repo access guidance, build/start commands, env **names** | VPS OS, Nginx, TLS, firewall, process supervisor |
| App fixes, migrations coordination, AfyaX API behaviour | Staging hostname DNS with commercial owner |
| UAT of app features + partner API handbook | Outbound IP documentation for AfyaX allowlists |
| Confirm health / integration after deploy | Monitoring, backups, patching on the box |

## 15. Quick reference — deploy acceptance checklist

☐ Ubuntu patched; deploy user + SSH keys
☐ Firewall configured
☐ Node 20.x installed

☐ npm lockfile install succeeds

☐ `npm run build` succeeds

☐ `.env` loaded (staging secrets)

☐ Postgres reachable; schema + RLS applied

☐ Redis configured

☐ PM2/systemd keeps process up across reboot

☐ Nginx + TLS + renew works

☐ `GET /api/v1/health` → ok

☐ AfyaX sandbox webhook path tested

☐ Logs + backups + monitoring on

☐ Written OK to keep Render as prod until cutover

---

## 16. Document control

| VERSION | DATE | NOTES |
|---|---|---|
| 1.0 | 15 Sep 2026 | First Sule-facing deploy guide from agreed VPS/staging plan + repo reality |

**Owner:** Alfred Mulinge (app) · coordinated with David Sule (infra)

**Commercial context:** UzimaX / Jugo Limited (Githuku)

---